NOVIQ / FREE LEARNING LIBRARY

Git & GitHub Starter Guide

Understand version control, save project history and collaborate with confidence.

GIT / 02 • PRACTICAL GUIDE • BEGINNER FRIENDLY

Clear explanations. Useful examples. Concepts you can apply.

01 / WHY VERSION CONTROL EXISTS

A project changes constantly. You add features, fix mistakes, experiment with ideas and sometimes need to return to an earlier working version. Version control records those changes so you can understand how a project evolved and collaborate without passing folders back and forth.

Git and GitHub are not the same

Git is a version control system that runs locally and records project history. **GitHub** is an online platform that hosts Git repositories and adds collaboration tools such as pull requests, issues and code review. Git can be used without GitHub.

02 / THE CORE TERMS

Repository: the project and its tracked history. **Commit:** a saved snapshot of selected changes. **Working tree:** the files you are currently editing. **Staging area:** the changes selected for the next commit. **Branch:** an independent line of development. **Remote:** a repository hosted elsewhere, often on a service such as GitHub.

03 / YOUR FIRST LOCAL WORKFLOW

Open a terminal in your project folder. These commands initialise Git, inspect changes, stage selected files and create a commit.

```
git init
git status
git add index.html style.css
git status
git commit -m "Create initial page"
```

Use git add . only when you have reviewed the files that will be included. A commit should represent one understandable unit of work. Write messages that explain the change, such as “Add responsive navigation” rather than “update”.

04 / CONNECTING TO A REMOTE

A remote repository gives you an online copy that can be shared with collaborators. Create an empty repository on your hosting platform, then connect your local repository and push your branch. The exact default branch name may be main or another configured name.

```
git remote add origin
git branch -M main
git push -u origin main
```

The repository URL is specific to your project. Do not copy a placeholder literally. Use the URL provided by your Git hosting service.

05 / PULL, PUSH AND CLONE

clone downloads a remote repository and its history to your computer. **pull** fetches remote changes and integrates them into your current branch. **push** sends your local commits to the remote repository. Before starting work, check whether teammates have pushed updates.

```
git clone
git pull
git push
```

If Git reports a conflict, it means it cannot automatically combine some changes. Open the affected files, review the conflict markers, decide what the final content should be, remove the markers, then stage and commit the resolved files.

06 / BRANCHES FOR SAFE EXPERIMENTS

A branch lets you work on a feature without immediately changing the main line of development. Create a branch with a clear name, make focused commits and merge it after review.

```
git switch -c feature/contact-form
# make and commit your changes
git switch main
git merge feature/contact-form
```

The comment line is explanatory, not a command you need to run. Teams may use pull requests to review a branch before merging it.

07 / COLLABORATING WITH PULL REQUESTS

A pull request proposes merging one branch into another. It gives teammates a place to inspect code, discuss decisions, run checks and request improvements. A useful pull request explains what changed, why it changed and how it was tested. Keep it focused so reviewers can understand it efficiently.

08 / KEEPING UNWANTED FILES OUT

Not every file belongs in version control. Generated build folders, temporary files, local environment settings and secrets should usually be excluded. A .gitignore file tells Git which untracked paths to ignore. Never commit passwords, private tokens or API secrets. If a secret is accidentally committed, removing it from the latest file is not enough; treat it as exposed and rotate it.

Example .gitignore

```
node_modules/
dist/
.env
.DS_Store
```

09 / FIXES AND USEFUL INSPECTION COMMANDS

Command	What it helps with
git status	See changed, staged and untracked files.
git log --oneline	Review recent commits in compact form.
git diff	Inspect unstaged changes.
git diff --staged	Inspect changes selected for the next commit.
git restore file	Discard unstaged changes in a file; use carefully.
git switch branch-name	Move to an existing branch.
git fetch	Download remote updates without integrating them.

10 / A SIMPLE DAILY ROUTINE

1. Start with git status.
2. Pull or fetch shared changes when appropriate.
3. Work on one task or branch at a time.
4. Review git diff before staging.
5. Stage only related files.
6. Commit with a clear message.
7. Push your branch and open a pull request when collaboration or review is needed.

11 / PRACTICE CHECKLIST

Try it: Create a small HTML project, initialise Git, make two commits, create a feature branch, change a heading, merge the branch and inspect the history. Then create a remote repository and push your work. Practise reading status and diff before memorising commands.

Safety note: Commands such as restore, reset and force push can discard work or rewrite shared history. Learn what they do before using them on an important project.

NOVIQ • Learn clearly. Build confidently.